Experiment 7 : Advanced ML Pipelines with 'From Scratch' Ensembles

Total Marks: 100

1. Learning Objectives

Upon successful completion of this assignment, students will be able to:

- Implement a complex, multi-stage preprocessing pipeline using `scikit-learn`, including date feature engineering, imputation, scaling, and categorical encoding.
- Integrate **Principal Component Analysis (PCA)** into a preprocessing pipeline to manage high-dimensionality.
- Implement custom data validation utilities, including a **K-Fold cross-validation** generator and a **mini-batch** generator, using only NumPy.
- Implement a high-level **Bagging** ensemble classifier (the meta-estimator) from scratch using NumPy/Pandas, using a simple `sklearn` model as a base estimator.
- Build a custom evaluation metric function for **F1-Score (Macro)** from scratch.
- Conduct hyperparameter tuning using their custom cross-validation loop.
- Rigorously evaluate the final model on a held-out validation set.

2. Introduction

This assignment moves beyond "plug-and-play" machine learning. You will tackle a realistic, "messy" bank marketing dataset (`train.csv`). Your first task is to build a robust preprocessing pipeline using `sklearn` to handle a mix of dates, categorical data, and numerical data. This pipeline will also include PCA to reduce the feature space.

The core challenge is to then **implement an ensemble learning algorithm (Bagging)** from scratch. While you may use a simple `sklearn` model (like `DecisionTreeClassifier`) as a *base learner*, your code must handle the ensemble logic: data bootstrapping (sampling with replacement), training multiple models, and aggregating their votes (prediction).

Finally, you will implement your own cross-validation and evaluation logic to tune and test your custom-built classifier, demonstrating a fundamental understanding of the entire ML pipeline.

Dataset: Predict the Success of Bank Telemarketing

3. Prerequisites

Ensure your Python environment has the following libraries installed:

```
pip install numpy pandas scikit-learn matplotlib seaborn
```

4. Experiment Tasks

You are required to build a complete machine learning pipeline. Follow the structured tasks below.

Task 1: Data Loading and Initial Setup (10 Marks)

1. **Load Data:** Load the `train.csv` file into a pandas DataFrame.
2. **Identify Target:** Separate the DataFrame into features `X` and the target variable `y` (the `target` column).
3. **Encode Target:** The `y` variable is categorical (`'yes'`, `'no'`). Convert it to a binary format (1 for `'yes'`, 0 for `'no'`).
4. **Create Hold-Out Set:** Perform a **single 70/30 split** on the data.
    - `X_train`, `y_train` (70% of the data): This is your **training set**. All cross-validation and model development will happen *only* on this set.
    - `X_val`, `y_val` (30% of the data): This is your **validation set**. You will *not* touch this set until the very final step to report your model's performance.
    - Use `random_state=42` for reproducibility.

Task 2: Advanced Preprocessing Pipeline (`sklearn`) (25 Marks)

Your goal is to build a single, unified `Pipeline` object that preprocesses the raw `X` data. This pipeline must be *fit* on `X_train` and used to *transform* both `X_train` and `X_val`.

1. **Date Feature Engineering:** The `last contact date` column is a string. Create a custom transformer (or use `FunctionTransformer`) that extracts the **`month`** and **`day_of_week`** as new features. (You can drop the original date column). These new features should be treated as categorical.
2. **Define Feature Lists:** Identify and create lists of column names for:
   - `numerical_features`: (e.g., `age`, `balance`, `duration`...)
   - `categorical_features`: (e.g., `job`, `marital`, `education`, and your new date features...)
3. **Create Transformers:**
   - **Numeric Transformer (`Pipeline`):**
     1. `SimpleImputer`: Fill missing numerical values with the **median**.
     2. `StandardScaler`: Scale all numerical features.
   - **Categorical Transformer (`Pipeline`):**
     1. `SimpleImputer`: Fill missing categorical values with the string **`'Missing'`** (or the mode).
     2. `OneHotEncoder`: Convert all categorical features to one-hot vectors. Use `handle_unknown='ignore'`.
4. **Combine with `ColumnTransformer`:** Create a `ColumnTransformer` that applies your numeric and categorical transformers to the correct feature lists.
5. **Integrate PCA:** Create a final, sequential `Pipeline` (let's call it `preprocessing_pipeline`) that contains two steps:
   a. `('preprocessor', your_column_transformer)`
   b. `('pca', PCA(n_components=0.95))`
      - This `n_components=0.95` will automatically select the number of components needed to explain 95% of the variance.
6. **Apply the Pipeline:**
   - `fit` the `preprocessing_pipeline` on `X_train`.
   - Transform `X_train` to get `X_train_processed`.
   - Transform `X_val` to get `X_val_processed`.
   - Print the `shape` of `X_train_processed` to see the final number of features after PCA.

Task 3: 'From Scratch' Utilities (NumPy) (20 Marks)

You must implement these helper functions using only NumPy/Pandas.

1. **F1-Macro Score:**
   - Create a function `get_f1_macro(y_true, y_pred)`.
   - This function must calculate the precision and recall for **both** class 0 and class 1.
   - From these, calculate the F1-score for class 0 and the F1-score for class 1.
   - Return the **average** of these two F1-scores. (Handle zero-division warnings gracefully).
2. **K-Fold Generator:**
   - Create a function `my_kfold(X, y, n_splits, shuffle=True, random_state=42)`.
   - This function should be a **generator**. It should `yield` `(X_train_fold, y_train_fold, X_test_fold, y_test_fold)` for each of the `n_splits`.
   - Use `np.random.permutation` (seeded with `random_state`) to shuffle the indices.
3. **Batch Generator:**
   - Create a function `get_bootstrap_sample(X, y, n_samples)`.
   - This function should perform **sampling with replacement** (bootstrapping) to select `n_samples` from `X` and `y`.
   - It should return `(X_batch, y_batch)`. Use `np.random.randint` or `np.random.choice`.

Task 4: 'From Scratch' Ensemble: Bagging (30 Marks)

Implement a Bagging classifier. It will be a Python class that manages the "ensemble logic" (bootstrapping, training, voting) but uses a simple `sklearn` estimator as its building block.

1. **Import Base Estimator:** You may use `from sklearn.tree import DecisionTreeClassifier` or `from sklearn.linear_model import LogisticRegression`.
2. **Create `MyBaggingClassifier` Class:**
   - `__init__(self, base_estimator_class, n_estimators, max_samples=1.0)`:
     - `base_estimator_class`: The *class* of the estimator (e.g., `DecisionTreeClassifier`).
     - `n_estimators`: The number of estimators to create.

    - `max_samples`: The fraction of the training data to use for each bootstrap sample (e.g., 1.0 = 100%).
    - Store an empty list for `self.estimators_`.
- `fit(self, X, y)`:
    - `n_samples_for_bootstrap = int(len(X) * self.max_samples)`
    - Loop `n_estimators` times:
        1. Call your `get_bootstrap_sample` function to get `(X_batch, y_batch)`.
        2. Create a *new instance* of your `base_estimator_class` (e.g., `estimator = self.base_estimator_class(max_depth=10, random_state=i)`).
        3. `fit` this new `estimator` on `X_batch` and `y_batch`.
        4. Append the trained `estimator` to `self.estimators_`.
- `predict(self, X)`:
    - Make predictions for `X` with *every* estimator in `self.estimators_`.
    - Store these predictions (e.g., in a 2D NumPy array of shape `(n_samples, n_estimators)`).
    - For each sample (row), find the **majority vote** (the mode) of the predictions.
    - Return the final 1D NumPy array of aggregated predictions.

Task 5: Integration, Tuning, and Final Evaluation (15 Marks)

This is your "main" script where you tie everything together.

1. **Tune Hyperparameters with CV:**
    - Choose a hyperparameter to tune for your `MyBaggingClassifier` (e.g., `n_estimators` in `[10, 30, 50]`).
    - Use your `my_kfold` generator on `X_train_processed` and `y_train` (e.g., with `n_splits=5`).
    - Write a loop: For each hyperparameter value:
        - For each `(X_fold_train, y_fold_train, X_fold_test, y_fold_test)` from `my_kfold`:
            - Instantiate `MyBaggingClassifier` with the current hyperparameter.
            - `fit` it on the fold's training data.

        - `predict` on the fold's test data.
        - Calculate the `get_f1_macro` score and store it.
    - Average the F1 scores for this hyperparameter.
- Select the hyperparameter value that gave the **best average F1-macro score**.

2. **Final Model Training:**
    - Instantiate `MyBaggingClassifier` with your **best** hyperparameters.
    - `fit` this final model on the **entire** `X_train_processed` and `y_train` dataset.
3. **Final Evaluation:**
    - Make predictions with your final model on `X_val_processed`.
    - Calculate the `get_f1_macro` score using `y_val` and your predictions.
    - Print a `classification_report` from `sklearn.metrics` for a detailed breakdown.

---

5. Submission Guidelines

Submit a single **.zip** archive containing:

1. **Source Code:** A single Jupyter Notebook (`.ipynb`) or multiple `.py` files containing all your code, clearly separated by task.
2. **PDF Report:** A formal report (`StudentID_Report.pdf`) that includes:
    - **Preprocessing:** Briefly explain your pipeline. State the original number of features and the final number of features after PCA.
    - **"From Scratch" Code:** Include the code snippets for your `get_f1_macro` , `my_kfold` , and `MyBaggingClassifier` class.
    - **CV Results:** Show the cross-validation results (e.g., a table or plot) of F1-macro vs. the hyperparameter you tuned.
    - **Final Performance:** State the final F1-Macro score achieved on the 30% hold-out validation set.
    - **Conclusion:** Briefly summarize your findings. What was the most challenging part? How did your "from scratch" model perform?